

GUÍA DE LABORATORIO

Laboratorio 3

Aplicación web pública por HTTP: construir, atacar, detectar, corregir y verificar

Duración	Modalidad	Equipos	Nivel
3 horas	Presencial / nube controlada	1–2 estudiantes	Intermedio

Resultado del laboratorio: Cada equipo publicará una aplicación web mínima por HTTP y sin autenticación en una instancia autorizada. Otro equipo realizará reconocimiento y validación técnica; el Blue Team observará el tráfico y los logs, aplicará controles iniciales y demostrará el cambio con evidencia reproducible.

Programa incremental corregido

Laboratorio	Construcción incremental	Ejercicio técnico principal
3	HTTP público sin autenticación	Reconocimiento, enumeración, tráfico en claro, logs y hardening básico
4	HTTPS + identidad + roles	Spoofing, sesiones, autenticación, autorización y certificados
5	DevSecOps y supply chain	SAST, SCA, secretos, SBOM, contenedores e IaC
6	Cloud Purple Team	DAST autenticado, WAF, observabilidad, respuesta e IA

Regla del reto: La solución será acumulativa.

1. Propósito y objetivos

Este laboratorio inicia el prototipo Secure Product Challenge y establece una línea base deliberadamente insegura, pero controlada. El objetivo no es “hackear por hackear”: cada prueba ofensiva debe producir telemetría, una explicación de riesgo, una corrección verificable y una evidencia útil para Blue Team.

- Desplegar una aplicación web mínima en una instancia Linux autorizada.
- Identificar la superficie de ataque con Nmap, curl y OWASP ZAP en modo pasivo.
- Demostrar por qué HTTP expone metadatos y contenido en tránsito.
- Relacionar evidencias con amenazas STRIDE, especialmente Information Disclosure y Tampering.
- Analizar access/error logs desde la perspectiva Blue Team.
- Aplicar hardening inicial sin anticipar autenticación ni HTTPS, que pertenecen al Laboratorio 4.
- Usar IA de forma responsable para explicar evidencia anonimizada, sin compartir secretos ni datos reales.

2. Reglas de seguridad y autorización

Alcance obligatorio: Solo se permite trabajar contra la IP o URL asignada y durante la ventana del laboratorio. Está prohibido escanear Internet, redes institucionales no asignadas, servicios de terceros o equipos de otros grupos antes de la autorización.

- Antes de ejecutar una prueba, documente: amenaza, hipótesis, comando, resultado esperado y evidencia que se capturará.
- Utilice únicamente cuentas, datos y tokens ficticios.
- No realice denegación de servicio, fuerza bruta, explotación destructiva, persistencia ni extracción de información real.
- El equipo defensor puede detener la prueba mediante la palabra de seguridad STOP-LAB.
- Cada comando debe conservar su timestamp, origen, destino y responsable.
- Anonimice IP públicas, nombres, tokens y cualquier dato sensible antes de usar una IA externa.

3. Arquitectura del ejercicio

Topología mínima: una estación Kali actúa como Red Team; una instancia Ubuntu Server aloja Nginx y la aplicación; el Blue Team monitorea el host y sus registros. Los equipos intercambian roles después de la primera ronda.

Componente	Función	Evidencia esperada
Kali Linux	Reconocimiento y validación autorizada	Comandos, timestamps, capturas y reporte ZAP
Ubuntu Server LTS	Host de la aplicación	Estado del servicio, firewall, configuración y logs
Nginx + sitio estático	Servicio HTTP sin autenticación	Respuesta HTTP, headers y recursos públicos
tcpdump / Wireshark	Visibilidad de red	PCAP filtrado del ejercicio
access.log / error.log	Telemetría defensiva	Eventos correlacionados con pruebas Red Team
IA autorizada u offline	Apoyo analítico opcional	Prompt anonimizado y validación humana

Variables que deben acordarse antes de comenzar:

```
export TARGET_IP=IP_ASIGNADA
export TARGET_URL=http://$TARGET_IP
export LAB_CIDR=CIDR_AUTORIZADO
date -u +%Y-%m-%dT%H:%M:%SZ
```

4. Preparación previa

Rol	Responsabilidad
Product/Builder	Publicar el sitio y mantener el repositorio.
Red Team	Plantear hipótesis y ejecutar pruebas autorizadas.
Blue Team	Capturar tráfico, revisar logs y construir detecciones.
Security Lead / Relator	Controlar el alcance, recolectar evidencias y presentar riesgos.

Materiales: Kali Linux, Ubuntu Server LTS, acceso sudo autorizado, Git, Nginx, Nmap, curl, tcpdump o Wireshark, OWASP ZAP y navegador. El docente debe asignar IP, CIDR y ventana de pruebas.

5. Agenda

Tiempo	Actividad	Resultado
0–20 min	Briefing, roles, alcance y criterios	Registro de autorización
20–55 min	Construcción y publicación HTTP	Aplicación accesible
55–75 min	DFD ligero + STRIDE	Hipótesis priorizadas
75–115 min	Ronda Red Team	Evidencias ofensivas
115–145 min	Ronda Blue Team	Logs, PCAP y detecciones
145–165 min	Corrección y hardening	Configuración mejorada
165–180 min	Retest, cierre y entrega	Comparación antes/después

6. Ejecución paso a paso

Fase A — Construir y publicar (Builder)

Paso 1. Verificar el host y registrar la línea base

```
hostnamectl
ip -br address
uname -a
date -u +%Y-%m-%dT%H:%M:%SZ
```

Guarde la salida como evidencia. Confirme que la IP coincide con la asignada y que no está trabajando sobre un servidor de producción.

Paso 2. Instalar y habilitar Nginx

```
sudo apt update
sudo apt install -y nginx
sudo systemctl enable --now nginx
sudo systemctl status nginx --no-pager
sudo ss -lntp | grep ':80'
```

Paso 3. Crear la aplicación mínima

Cree un sitio sin autenticación que muestre únicamente información ficticia. El siguiente contenido se escribe en `/var/www/muvautomation/index.html` mediante el editor de texto aprobado por el docente:

```
<!doctype html>
<html lang="es">
<head><meta charset="utf-8"><title>MuvAutomation Lab</title></head>
<body>
  <h1>MuvAutomation Asset Portal</h1>
  <p>Environment: LAB</p>
  <p>Owner: Blue Team</p>
  <a href="/public-inventory.txt">Inventario público de demostración</a>
</body>
</html>
```

Cree también public-inventory.txt con tres activos ficticios, por ejemplo WEB-LAB-01, API-LAB-01 y DB-LAB-01. No incluya credenciales, correos reales ni direcciones internas reales.

Paso 4. Configurar el virtual host

```
server {
  listen 80 default_server;
  listen [::]:80 default_server;
  root /var/www/muvautomation;
  index index.html;
  server_name _;
  location / { try_files $uri $uri/ =404; }
```

Guarde el bloque como /etc/nginx/sites-available/muvautomation, habilítelo, retire el sitio default si corresponde y valide la sintaxis:

```
sudo ln -s /etc/nginx/sites-available/muvautomation /etc/nginx/sites-enabled/muvautomation
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx
curl -i http://127.0.0.1/
```

Paso 5. Limitar el firewall al segmento del laboratorio

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow from "$LAB_CIDR" to any port 80 proto tcp
sudo ufw allow OpenSSH
sudo ufw enable
sudo ufw status numbered
```

Punto de control: El docente verifica la URL, el alcance del firewall y que el contenido sea completamente ficticio antes de autorizar las pruebas.

Fase B — Modelar antes de atacar (todos)

Paso 6. Dibujar el flujo mínimo

Represente: Usuario/Kali → Internet o red del laboratorio → Nginx → archivos del sitio. Marque como límites de confianza la entrada al servidor y el cambio entre red y aplicación.

Paso 7. Formular hipótesis STRIDE

ID	STRIDE	Hipótesis técnica	Validación
H1	Information Disclosure	HTTP permite observar contenido y rutas en tránsito.	Captura PCAP filtrada.
H2	Information Disclosure	Headers y respuestas revelan tecnología o recursos.	curl -I y ZAP pasivo.
H3	Repudiation	Sin correlación temporal, el equipo no atribuye solicitudes.	Comparar comando con access.log.
H4	Tampering	Sin TLS, un intermediario podría alterar tráfico; no se ejecutará MITM.	Demostrar ausencia de protección, sin interceptar terceros.

Fase C — Atacar de manera controlada (Red Team)

Paso 8. Reconocimiento de puerto y servicio

```
mkdir -p evidence/red
date -u +%Y-%m-%dT%H:%M:%SZ | tee evidence/red/start.txt
nmap -Pn -sV -p 80 "$TARGET_IP" -oA evidence/red/nmap_port80
curl -i "$TARGET_URL/" | tee evidence/red/curl_home.txt
curl -I "$TARGET_URL/public-inventory.txt" | tee evidence/red/curl_headers.txt
```

Interprete los resultados: puerto, protocolo, producto identificado, status codes, headers y rutas visibles. No amplíe el rango de puertos sin autorización adicional.

Paso 9. Construir el inventario de superficie de ataque

Elemento	Dato observado	Riesgo / pregunta
Host/IP	Completar	¿Está limitado al CIDR autorizado?
Puerto	80/TCP	¿Por qué el tráfico no tiene confidencialidad?
Servidor	Completar	¿Se revela versión?
Ruta /	200 esperado	¿Expone información innecesaria?
Archivo público	Completar	¿Qué metadatos entrega?

Paso 10. Inspección pasiva con OWASP ZAP

1. Abra ZAP y seleccione Manual Explore.

2. Ingrese únicamente TARGET_URL.
3. Navegue por la página y el archivo público.
4. Revise Alerts y Sites sin ejecutar Active Scan.
5. Exporte el reporte HTML y registre cada header ausente como observación, no automáticamente como vulnerabilidad explotable.

Paso 11. Demostrar visibilidad de HTTP

En el host, el Blue Team inicia una captura limitada. Después, Red Team solicita la página y el archivo ficticio:

```
# Blue Team, en el servidor
sudo timeout 60 tcpdump -i any -nn -s0 -w /tmp/lab3-http.pcap 'tcp port 80'

# Red Team, durante esos 60 segundos
curl "$TARGET_URL/"
curl "$TARGET_URL/public-inventory.txt"
```

Abra el PCAP con Wireshark y aplique el filtro http. Documente qué información puede observarse. No capture tráfico de usuarios ajenos al laboratorio.

Fase D — Detectar y correlacionar (Blue Team)

Paso 12. Revisar telemetría del servidor

```
sudo tail -n 50 /var/log/nginx/access.log
sudo tail -n 30 /var/log/nginx/error.log
sudo journalctl -u nginx --since '20 minutes ago' --no-pager
```

Correlacione IP origen, timestamp, método, ruta, status code, bytes y User-Agent con cada comando Red Team.

Paso 13. Crear una detección reproducible

```
sudo awk '$9 ~ /404/ {print $1, $4, $7, $9, $12}' /var/log/nginx/access.log | sort |
uniq -c | sort -nr | head

sudo grep -E 'nmap|curl|ZAP' /var/log/nginx/access.log | tail -n 30
```

La regla de laboratorio genera una señal cuando una misma IP produce cinco o más respuestas 404 en cinco minutos. El equipo debe explicar sus limitaciones y falsos positivos; no es obligatorio automatizar el bloqueo.

Paso 14. Elaborar una línea de tiempo Purple Team

Hora UTC	Acción Red Team	Evidencia Blue Team	Conclusión
Completar	Nmap port 80	access.log puede no registrar SYN scan	Diferenciar red vs. aplicación
Completar	GET /	200 en access.log	Correlación confirmada

Hora UTC	Acción Red Team	Evidencia Blue Team	Conclusión
Completar	GET archivo	200 y PCAP	Contenido visible por HTTP
Completar	Ruta inexistente	404 en access.log	Detección validada

Fase E — Corregir sin adelantar el Laboratorio 4 (Blue Team)

Paso 15. Aplicar hardening inicial de Nginx

Agregue dentro del bloque server:

```
server_tokens off;
add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "DENY" always;
add_header Referrer-Policy "no-referrer" always;
autoindex off;
location ~ /\. { deny all; }
sudo nginx -t
sudo systemctl reload nginx
curl -I "$TARGET_URL/"
curl -i "$TARGET_URL/.git/config"
```

Límite pedagógico: HTTP seguirá siendo inseguro para confidencialidad e integridad. Ese riesgo queda explícitamente abierto para ser corregido con certificados, HTTPS, identidad y roles en el Laboratorio 4.

Paso 16. Reducir exposición de contenido

Revise el inventario público y elimine cualquier dato que no sea estrictamente necesario. Conserve una versión antes/después en Git y justifique el cambio.

Fase F — Verificar y cerrar (Purple Team)

Paso 17. Repetir exactamente las pruebas

```
mkdir -p evidence/retest
nmap -Pn -sV -p 80 "$TARGET_IP" -oA evidence/retest/nmap_port80
curl -I "$TARGET_URL/" | tee evidence/retest/headers_after.txt
curl -i "$TARGET_URL/.git/config" | tee evidence/retest/hidden_path.txt
```

Compare los resultados. El servicio puede continuar identificándose como HTTP, pero la versión no debe exponerse en banners, los headers definidos deben aparecer y la ruta oculta debe ser denegada.

Paso 18. Etiquetar la entrega

```
git status
git add .
git commit -m "lab3: public HTTP baseline, telemetry and initial hardening"
git tag lab-3
git show --stat --oneline lab-3
```

7. Uso responsable de IA

La IA funciona como copiloto analítico, no como autoridad. El estudiante debe verificar cada afirmación contra comandos, logs, PCAP o configuración.

Prompt sugerido:

Prompt: Analiza estos eventos Nginx anonimizados. Construye una línea de tiempo, separa hechos de inferencias, mapea cada observación a STRIDE, propone tres hipótesis defensivas y señala qué evidencia adicional falta. No inventes IP, CVE ni acciones ejecutadas.

- No enviar PCAP completos a servicios públicos.
- Sustituir IP, hostname, usuario, token y ruta sensible.
- Adjuntar prompt y respuesta en la evidencia.
- Marcar cualquier alucinación detectada.
- La calificación depende de la validación humana, no de la extensión del texto generado.

8. Entregables

- Repositorio acumulativo con tag lab-3.
- README con arquitectura, variables y procedimiento de reproducción.
- DFD ligero con al menos dos límites de confianza.
- Tabla STRIDE con mínimo cuatro hipótesis.
- Carpeta evidence/red con Nmap, curl y reporte pasivo ZAP.
- Carpeta evidence/blue con logs relevantes, PCAP controlado y regla de detección.
- Comparación antes/después de las correcciones.
- Registro de riesgos: corregido, mitigado, aceptado o pendiente para Lab 4.
- Reflexión individual de máximo 250 palabras.

9. Estructura esperada del repositorio

```

muvautomation-secure-challenge/
├─ app/
├─ nginx/
├─ diagrams/dfd-lab3.png
├─ evidence/red/
├─ evidence/blue/
├─ reports/zap-passive/
├─ risk-register.md
└─ README.md

```

10. Criterios de evaluación

Criterio	Peso	Evidencia de logro
Construcción reproducible	20%	Aplicación disponible, configuración versionada y alcance correcto.
Red Team técnico	20%	Hipótesis previa, Nmap/curl/ZAP pasivo y evidencia interpretada.
Blue Team técnico	20%	PCAP controlado, logs correlacionados y regla evaluada.
Threat Modeling	15%	DFD, trust boundaries y STRIDE conectados con evidencia.
Corrección y retest	15%	Cambio verificable y comparación antes/después.
Calidad y ética	10%	Trazabilidad, datos ficticios, alcance y uso responsable de IA.

Condición de aprobación: Una captura aislada no constituye evidencia suficiente. Cada hallazgo debe incluir hipótesis, comando o acción, timestamp, resultado, interpretación, corrección y retest.

11. Checklist de cierre

- ☐ La IP objetivo fue autorizada.
- ☐ No se incluyeron datos reales.
- ☐ El servicio responde por HTTP desde el segmento permitido.
- ☐ Red Team conservó comandos y timestamps.
- ☐ Blue Team correlacionó al menos tres eventos.

- ☐ El PCAP solo contiene tráfico del laboratorio.
- ☐ Se aplicaron headers y reducción de exposición.
- ☐ Se ejecutó el retest.
- ☐ Los riesgos pendientes quedaron registrados para Lab 4.
- ☐ El tag lab-3 existe y apunta a la entrega final.

12. Preguntas de análisis

- ¿Qué pudo observar el Red Team sin explotar ninguna vulnerabilidad?
- ¿Qué pruebas de red no aparecieron en access.log y por qué?
- ¿Qué control aplicado reduce exposición, pero no resuelve el riesgo de HTTP?
- ¿Qué datos necesitaría Blue Team para distinguir curl legítimo de una actividad sospechosa?
- ¿Qué amenaza STRIDE debe priorizarse en el Laboratorio 4?
- ¿Qué conclusión propuesta por la IA no pudo comprobarse directamente?

13. Resultado esperado

Al finalizar, el equipo debe demostrar el ciclo completo Diseñar → Construir → Atacar → Detectar → Corregir → Verificar. El producto continuará siendo deliberadamente incompleto: seguirá sin autenticación y sin HTTPS, creando la base técnica para el Laboratorio 4, donde se trabajarán certificados, identidad, sesiones, roles y escenarios de Spoofing.